

Interview Preparation Guide: Event Management System

1. Project Overview & Core Purpose

- **Project Name:** CampusConnect (Event Management System)
 - **Purpose:** To automate the lifecycle of campus events—from creation by clubs to student registration, secure OTP-based entry, and administrative reporting (Excel exports).
 - **Problem Solved:** Replaced manual, paper-based, or fragmented registration systems with a centralized platform that ensures data integrity and security.
- 2. Technical Stack (The "What" and "Why")

Technology	Purpose	Why this choice?
React 19 + Vite	Frontend Framework	Fast rendering, component reusability, and industry-standard for modern web apps. Vite provides a much faster development experience than Create React App.
Node.js + Express	Backend Runtime/Framework	Non-blocking I/O makes it highly scalable for concurrent registrations. Using JavaScript on both ends (Fullstack) speeds up development.
MySQL	Database	Relational structure is perfect for managing complex relationships between Users, Clubs, Events, and Registrations.
JWT (JSON Web Token)	Authentication	Stateless authentication allows the server to verify users without storing session data in memory, making it scalable and secure.
Tailwind CSS	Styling	Utility-first approach allows for rapid UI development and ensures a modern, responsive design without writing thousands of lines of custom CSS.
Framer Motion	Animations	Adds a "premium" feel with smooth transitions, making the app feel alive and professional.
Nodemailer	Email Service	Used to send automated "Entry Passes" with unique OTPs directly to students' emails.
XLSX (SheetJS)	Data Export	Automatically syncs database records into Excel sheets for administrators to use during event check-ins.

3. System Design Patterns & Advanced Concepts

During the interview, you can talk about these design patterns to show your depth of knowledge:

- **1. Singleton Pattern (Implemented)**
 - **Where:** In backend/config/db.js (Database Pool) and web/src/utils/api.js (Axios Instance).
 - **Definition:** Ensuring a class/object has only one instance throughout the application.
 - **Why:** Creating a new database connection for every request is expensive and slow. Using a single "Pool" instance saves resources.
- **2. Idempotency (Implemented at Application Level)**
 - **Where:** In registrationController.js (Registration verification).
 - **Definition:** An operation that can be performed multiple times without changing the result beyond the initial application.
 - **How:** Before saving a new registration, the code checks if the RollNumber already exists for that EventID. This prevents duplicate entries if a user clicks "Submit" twice or if the network retries the request.
- **3. Factory Pattern (Proposed for Upgrade)**
 - **Explanation:** Currently, you create objects directly. A **Notification Factory** could be implemented to decide whether to send an Email, SMS, or In-App notification based on the event type or user preference.
- **4. ETagging & Optimistic Locking (Proposed for Upgrade)**
 - **Explanation:** When two admins edit the same event details at once, the last one to save wins (Last-Write-Wins). Implementing **ETags** (versioning) would allow the system to detect if the data changed while the admin was editing, preventing accidental overwrites.

4. Project Development Phases

Explain these as the steps you took to build the project:

1. **Requirement Analysis:** Identified stakeholders (Super Admin, Club Admins, Students) and their specific needs.
2. **Database Modeling:** Designed the schema with normalization to ensure no data redundancy.
3. **API Development (Backend First):** Built the RESTful API endpoints for Auth, Event Management, and Registration.

4. **Frontend Integration:** Developed the React UI, implemented Protected Routes (RBAC - Role-Based Access Control), and connected to the backend using Axios.
5. **Security & Hardening:** Implemented password hashing (Bcrypt), JWT security, and Multer for secure file uploads.
6. **Testing & Optimization:**
 - **Postman Testing:** Verified all API endpoints.
 - **Caching:** Implemented a 5-minute cache for student verification data (Excel reads) to boost performance.
 - **Background Tasks:** Set the Excel sync to run in the background so it doesn't slow down the user's registration experience.

5. Common Interview Questions (And How to Answer)

- **Q: How do you secure user passwords?**
 - **A:** "I use bcryptjs to hash passwords with a salt before storing them in MySQL. This ensures that even if the database is compromised, the actual passwords remain secure."
- **Q: How do you handle file uploads (Posters/Screenshots)?**
 - **A:** "I use Multer as middleware to handle multipart/form-data. Files are stored on the server with unique timestamps to prevent name collisions, and the file path is stored in the database."
- **Q: What happens if the database goes down?**
 - **A:** "The database connection pool in db.js is configured to handle connection limits. In a production environment, I would add retry logic and more robust error handling to inform the user gracefully."

"I built CampusConnect, a full-stack Event Management System designed to solve the chaos of manual event registrations.

On the Backend, I chose Node.js and Express for its scalability, paired with MySQL for relational data integrity. I implemented a Singleton pattern for the database connection pool to optimize resource usage. For security, I used JWT-based stateless authentication and Bcrypt for password hashing.

On the Frontend, I used React 19 with Vite for a high-performance user experience. I focused heavily on UX/UI using Tailwind CSS and Framer Motion to ensure the platform feels modern and premium.

One key technical challenge I solved was Idempotency in registrations. I built logic to verify roll numbers against an event ID before allowing a record to be created, preventing duplicate data. I also implemented a background task to sync database records with Excel files using the SheetJS library, allowing administrators to have offline access to participant lists."